



## **UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA**

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

# **GUÍA DIDÁCTICA ACTUALIZADA 2026** **ASEGURAMIENTO DE LA CALIDAD DEL** **SOFTWARE**

Código del curso: 048

### **Propuesta de actualización curricular**

Quality Engineering, automatización, calidad continua y evaluación responsable de sistemas con IA

### **Ciclo académico 2026**

Versión preparada el 18 de julio de 2026

# Contenido

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Contenido</b>                                                    | <b>2</b>  |
| <b>1. Datos generales</b>                                           | <b>4</b>  |
| <b>2. Presentaci n de la gu a</b>                                   | <b>4</b>  |
| <b>3. Justificaci n de la actualizaci n</b>                         | <b>4</b>  |
| 3.1 Hallazgos en la gu a institucional y el curso 2025 . . . . .    | 4         |
| 3.2 Qu se conserva, qu se reubica y qu se incorpora . . . . .       | 5         |
| 3.3 L mites con otros cursos . . . . .                              | 5         |
| <b>4. Fundamentaci n o intenci n educativa</b>                      | <b>5</b>  |
| <b>5. Competencia macro</b>                                         | <b>6</b>  |
| <b>6. Resultados de aprendizaje del curso</b>                       | <b>6</b>  |
| <b>7. Enfoque tecnol gico 2026</b>                                  | <b>6</b>  |
| <b>8. Progresi n por m dulos</b>                                    | <b>7</b>  |
| <b>9. Metodolog a de aprendizaje</b>                                | <b>7</b>  |
| 9.1 Estructura recomendada para una sesi n de 120 minutos . . . . . | 7         |
| 9.2 Variantes y accesibilidad . . . . .                             | 8         |
| <b>10. Proyecto integrador del curso</b>                            | <b>15</b> |
| 10.1 Hitos progresivos . . . . .                                    | 15        |
| 10.2 Requisitos m nimos del repositorio . . . . .                   | 15        |
| 10.3 Quality gate recomendado . . . . .                             | 15        |
| <b>11. Evaluaci n</b>                                               | <b>16</b> |
| 11.1 Principios de evaluaci n . . . . .                             | 16        |
| 11.2 R brica est ndar para tareas de 3 puntos . . . . .             | 16        |
| 11.3 Formato de entrega de tareas breves . . . . .                  | 16        |
| <b>12. Pol tica de uso de inteligencia artificial</b>               | <b>17</b> |
| 12.1 Registro AI_USAGE.md . . . . .                                 | 17        |
| 12.2 Criterios de defensa . . . . .                                 | 17        |

13. Pruebas de sistemas habilitados con IA

18

14. Organizaci n recomendada en Canvas LMS

19

14.1 Plantilla para cada bloque tem tico . . . . .

19

14.2 Convenciones de dise o y operaci n . . . . .

19

14.3 Checklist de publicaci n . . . . .

20

15. Matriz de actualizaci n respecto a la gu a anterior

20

15.1 Matriz de mejora respecto al curso impartido en 2025 . . . . .

21

16. Bibliograf a y fuentes oficiales modernas

22

16.1 Protocolo de revisi n anual . . . . .

22

17. Anexo A - Entorno m nimo recomendado

23

18. Anexo B - Checklist para una evidencia de calidad

23

19. Anexo C - Glosario esencial

24

**Criterio de organizaci n**

La gu a se organiza por m dulos y bloques tem ticos. No fija semanas ni fechas: la coordinaci n acad mica podr adaptar el ritmo sin alterar la progresi n, los resultados de aprendizaje ni las evidencias m nimas.

# 1. Datos generales

|                             |                                                                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nombre del curso            | Aseguramiento de la Calidad del Software                                                                                                                       |
| Código                      | 048                                                                                                                                                            |
| Prerrequisito institucional | 175 créditos, según la guía institucional base.                                                                                                                |
| Población objetivo          | Estudiantes de Ingeniería en Sistemas de Información y Ciencias de la Computación con fundamentos de programación, bases de datos, Web y control de versiones. |
| Naturaleza                  | Curso teórico-práctico, orientado a Quality Engineering y a la toma de decisiones basada en riesgo y evidencia.                                                |
| Duración                    | Variable según calendario institucional. Cada sesión planificada cubre 120 minutos completos.                                                                  |
| Organización                | Tres módulos progresivos y un proyecto integrador transversal. Las evaluaciones principales se realizan al cierre de cada módulo.                              |
| Lenguaje transversal        | TypeScript sobre el ecosistema JavaScript. Se priorizan contratos tipados, automatización reproducible y herramientas de uso profesional.                      |
| Modalidad                   | Adaptable a presencial, virtual o híbrida, con repositorio Git, Canvas LMS y laboratorios reproducibles.                                                       |
| Versión                     | Actualización curricular 2026. Las versiones concretas de herramientas se verifican al iniciar cada edición del curso.                                         |

## 2. Presentación de la guía

Esta guía actualiza el curso institucional de Aseguramiento de la Calidad del Software para un contexto profesional en el que la calidad se diseña, automatiza, observa y gobierna durante todo el ciclo de vida. Conserva los fundamentos duraderos de aseguramiento, control, verificación, validación, diseño de pruebas y gestión de defectos; los conecta con prácticas actuales de desarrollo, entrega y operación.

La propuesta también toma como insumo el curso impartido en 2025. Esa experiencia ya incorporó una ruta práctica valiosa con TypeScript, Vitest, GitHub Actions, PostgreSQL, Docker, Postman, Cypress y Maestro. La actualización mantiene dichas fortalezas, reduce la dependencia de actividades centradas solo en la herramienta y completa capacidades que el mercado espera de un perfil moderno de calidad.

### Principio rector

Los conceptos constituyen el currículo; las herramientas constituyen el laboratorio. El estudiante debe poder trasladar una estrategia de calidad a otras tecnologías, justificar el nivel de prueba elegido y producir evidencia reproducible.

## 3. Justificación de la actualización

### 3.1 Hallazgos en la guía institucional y el curso 2025

La guía institucional establece una base válida, pero su dosificación depende de semanas fijas, bibliografía envejecida y una secuencia que culmina en Robot Framework como herramienta destacada.

La ruta 2025 moderniza la práctica, aunque comenzó con configuración de herramientas antes de consolidar criterios de aceptación, análisis de riesgos y técnicas sistemáticas de diseño.

Postman se utilizó para pruebas funcionales de API y para carga; en 2026 conviene separar esos objetivos y utilizar k6 para modelos, métricas y umbrales de rendimiento.

Las tareas alternaron entre 3 y 4 puntos. La nueva guía normaliza la mayoría de las evidencias breves a 3 puntos y proporciona una sola rbrica institucional reutilizable.

No aparecen como rutas explícitas: contratos con OpenAPI/Pact, mutation testing, accesibilidad, regresión visual, seguridad basada en OWASP ASVS, observabilidad, control de flakiness y pruebas de sistemas con IA.

### 3.2 Qué se conserva, qué se reubica y qué se incorpora

| Decisión                | Contenido                                                                                                    | Aplicación 2026                                                                       |
|-------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Conservar               | QA/QC, verificación y validación, niveles, tipos, diseño, defectos.                                          | Se presentan con terminología consistente y decisiones basadas en riesgo.             |
| Conservar y profundizar | TypeScript, Vitest, GitHub Actions, PostgreSQL, Docker, Postman, Cypress y Maestro.                          | Forman una ruta integrada con repositorio, artefactos, contratos y quality gates.     |
| Mover a referencia      | GitHub Desktop, Selenium, JMeter, Robot Framework, Jasmine y Jest.                                           | Se comparan cuando aportan contexto; no ocupan el espacio práctico.                   |
| Incorporar              | Testcontainers, OpenAPI, Pact, Playwright, k6, Stryker, OWASP ASVS, WCAG, visual regression y OpenTelemetry. | Cubren integración real, contratos, E2E moderno, calidad no funcional y diagnóstico.  |
| Incorporar              | IA asistida y pruebas de sistemas con IA.                                                                    | Se exige trazabilidad, validación humana, evaluación repetible y protección de datos. |

### 3.3 Límites con otros cursos

| Área                    | Tratamiento en este curso                                                          | Profundización en otros cursos                                                |
|-------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Programación            | Código suficiente para diseñar y automatizar pruebas.                              | Sintaxis, algoritmos, patrones y arquitectura general.                        |
| Bases de datos          | Aislamiento, fixtures, migraciones y validación de integración con PostgreSQL.     | Modelado, optimización, administración e internals del motor.                 |
| Desarrollo Web/móvil    | Uso del producto como objeto de prueba, con énfasis en comportamiento y evidencia. | Construcción completa de interfaces, APIs y aplicaciones móviles.             |
| Seguridad               | Verificación de controles y riesgos frecuentes basada en OWASP ASVS.               | Pentesting avanzado, criptografía, gobierno y respuesta a incidentes.         |
| DevOps/Cloud            | Pipelines de pruebas, servicios efímeros, artefactos y quality gates.              | Infraestructura, despliegue, operación, SRE y plataforma.                     |
| Inteligencia artificial | Evaluación de comportamiento, conjuntos de prueba y riesgos de sistemas con IA.    | Construcción y entrenamiento de modelos, MLOps e investigación especializada. |

## 4. Fundamentación o intención educativa

El curso forma criterio para prevenir defectos, seleccionar técnicas, automatizar verificaciones y comunicar riesgos. La práctica no se limita a obtener un resultado verde: cada prueba debe responder una pregunta de calidad, usar un oráculo razonable, controlar sus datos y producir evidencia útil para decidir.

La progresión combina aprendizaje activo, práctica deliberada, casos, revisión entre pares y un proyecto acumulativo. Se adopta un enfoque de shift-left y shift-right: calidad temprana en requisitos, diseño y código; y calidad operacional mediante telemetría, experimentación controlada y aprendizaje de incidentes.

## 5. Competencia macro

### Competencia general

Diseñar, ejecutar, automatizar y comunicar una estrategia de calidad de software basada en riesgos y evidencia, integrando pruebas funcionales y no funcionales, calidad continua, observabilidad y uso responsable de inteligencia artificial para favorecer productos confiables, seguros, accesibles y mantenibles.

## 6. Resultados de aprendizaje del curso

- Distinguir calidad de producto y proceso, aseguramiento, control, verificación, validación, error, defecto, fallo y riesgo.
- Traducir requisitos y criterios de aceptación en condiciones de prueba observables y trazables.
- Aplicar clases de equivalencia, valores límite, tablas de decisión, transiciones de estado, pairwise y técnicas estructurales básicas.
- Construir pruebas unitarias y de componentes en TypeScript con Vitest, dobles de prueba, cobertura y mutation testing.
- Diseñar pruebas de integración reproducibles con PostgreSQL, Docker y Testcontainers, controlando migraciones, datos y aislamiento.
- Validar APIs con Postman y OpenAPI, y verificar contratos de consumidor/proveedor con Pact.
- Automatizar flujos End-to-End web con Playwright y comparar decisiones con Cypress; automatizar flujos móviles con Maestro cuando el proyecto lo requiera.
- Evaluar rendimiento con k6, seguridad con OWASP ASVS, accesibilidad con WCAG 2.2 y cambios visuales mediante regresión visual.
- Integrar pruebas en GitHub Actions con reportes, artefactos, ejecución paralela, umbrales y diagnóstico de pruebas inestables.
- Interpretar cobertura, mutation score, duración, flakiness, defectos escapados, logs, métricas y trazas para sustentar decisiones.
- Utilizar IA como asistente con trazabilidad y validación humana, y evaluar sistemas con IA considerando variabilidad, seguridad y calidad de resultados.
- Comunicar hallazgos mediante reportes de defectos, evidencia reproducible, video breve y defensa técnica.

## 7. Enfoque tecnológico 2026

### Regla de versiones

No se fijan versiones de runtimes o herramientas en el currículo. Al iniciar cada edición se seleccionan versiones estables o LTS compatibles, se registran en el repositorio base y se valida la documentación oficial.

| Capa                   | Ruta principal                                  | Uso pedagógico                                                           |
|------------------------|-------------------------------------------------|--------------------------------------------------------------------------|
| Lenguaje y repositorio | TypeScript, Node.js, Git y GitHub               | Tipos, modularidad, historial, revisión y colaboración.                  |
| Unidad/componentes     | Vitest                                          | AAA, fixtures, mocks, spies, snapshots, cobertura y calidad de la suite. |
| Integración            | PostgreSQL, Docker y Testcontainers             | Dependencias reales, efímeras y reproducibles.                           |
| API y contratos        | Postman, OpenAPI 3.2 y Pact                     | Exploración, esquemas, compatibilidad y confianza entre servicios.       |
| Web y móvil            | Playwright; Cypress comparativo; Maestro        | E2E, trazas, evidencia visual, multi-navegador y flujos móviles.         |
| No funcional           | k6, OWASP ASVS 5.0, WCAG 2.2, visual regression | Rendimiento, seguridad, accesibilidad y cambios de interfaz.             |

| Capa                    | Ruta principal                                                     | Uso pedagógico                                                                   |
|-------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Calidad continua        | GitHub Actions                                                     | Quality gates, matrices, caché, artefactos, seguridad de secretos y diagnóstico. |
| Efectividad y operación | Stryker y OpenTelemetry                                            | Mutation score, trazas, métricas, logs y correlación de evidencia.               |
| Sistemas con IA         | Conjuntos de evaluación, rbricas, red teaming y supervisión humana | Evaluación repetible de precisión, robustez, seguridad y variabilidad.           |

## 8. Progreso en módulos

| Módulo                                    | Pregunta rectora                                                                                    | Capacidades                                                                                     | Producto de cierre                                                        |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1. Fundamentos y calidad desde el código  | ¿Qué significa calidad y cómo diseñamos pruebas que detecten riesgos reales?                        | Estrategia, criterios, técnicas, pruebas unitarias, TDD, cobertura, mutación y CI inicial.      | Estrategia de prueba y suite unitaria defendible con pipeline.            |
| 2. Integración y automatización de flujos | ¿Cómo obtenemos confianza entre componentes, datos, APIs y experiencias de usuario?                 | Integración real, Testcontainers, API, OpenAPI, Pact, E2E web, visual regression y CI ampliada. | Ruta automatizada desde integración hasta E2E con contratos y artefactos. |
| 3. Calidad avanzada y operación           | ¿Cómo evaluamos atributos no funcionales y sistemas variables en condiciones cercanas a producción? | k6, OWASP, WCAG, mévil, observabilidad, métricas, flakiness y pruebas de IA.                    | Quality gate integral y defensa del proyecto con evaluación de riesgos.   |

## 9. Metodología de aprendizaje

Aprendizaje basado en problemas y riesgos: cada laboratorio parte de una pregunta de calidad y un contexto de decisión.

Demostración breve seguida de práctica guiada, reto independiente y explicación del razonamiento.

Proyecto acumulativo: el mismo producto evoluciona y evita ejercicios desconectados sin contexto.

Revisión de código y pruebas entre pares con listas de verificación y retroalimentación accionable.

Evidencia reproducible: repositorio, ejecución, reporte, artefactos y explicación breve.

IA con supervisión: puede acelerar exploración y borradores, pero no sustituye comprensión, verificación ni autoría responsable.

### 9.1 Estructura recomendada para una sesión de 120 minutos

| Momento            | Min | Propósito                                                              | Evidencia                             |
|--------------------|-----|------------------------------------------------------------------------|---------------------------------------|
| Activación         | 10  | Recuperar conocimiento, analizar un fallo o plantear el riesgo.        | Pregunta diagnóstica o hipótesis.     |
| Concepto focal     | 20  | Introducir un modelo, técnica o decisión con ejemplos.                 | Notas, mapa o criterio de éxito.      |
| Demostración       | 20  | Modelar el proceso y verbalizar decisiones y errores frecuentes.       | Ejecución comentada y artefactos.     |
| Laboratorio guiado | 40  | Aplicar la técnica con apoyo progresivamente reducido.                 | Commit funcional y pruebas.           |
| Reto y revisión    | 20  | Extender el caso, comparar soluciones o revisar evidencia entre pares. | Cambio justificable o hallazgo.       |
| Cierre             | 10  | Consolidar, registrar dudas y conectar con el proyecto.                | Exit ticket y siguiente acción.       |
| Total              | 120 | Sesión completa.                                                       | Evidencia de aprendizaje verificable. |

## 9.2 Variantes y accesibilidad

En sesiones conceptuales puede ampliarse el análisis de casos y reducir la demostración; en clínicas de proyecto puede dedicarse hasta 70 minutos a diagnóstico y revisión. Los materiales deben ofrecer estructura semántica, contraste, texto alternativo, subtítulos o transcripción y alternativas equivalentes cuando exista una necesidad de accesibilidad documentada.



## MÓDULO 1

## Fundamentos y calidad desde el código

Del lenguaje común a una suite que detecta cambios y participa en CI.

## Bloque 1. Calidad, riesgo y estrategia de pruebas

|                          |                                                                                                                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Construir un lenguaje común y decidir qué probar, por qué, cuándo y con qué profundidad.                                                                                                  |
| <b>Resultados</b>        | Distingue QA/QC, V&V; error/defecto/fallo; prioriza riesgos y define objetivos de prueba.                                                                                                 |
| <b>Contenidos</b>        | Calidad de producto/proceso; Quality Engineering; atributos ISO/IEC 25010; siete principios; niveles y tipos; shift-left/right; riesgo, probabilidad, impacto, estrategia y trazabilidad. |
| <b>Demostración</b>      | Análisis de una historia ambigua y construcción de un mapa riesgo-objetivo-evidencia.                                                                                                     |
| <b>Práctica</b>          | Evaluar un producto pequeño, identificar riesgos y redactar una estrategia de una página.                                                                                                 |
| <b>Tecnologías</b>       | Canvas, GitHub Issues/Projects o tablero equivalente; plantilla de estrategia.                                                                                                            |
| <b>Canvas LMS</b>        | Página de bienvenida, diagnóstico, glosario, caso inicial, plantilla y foro de riesgos.                                                                                                   |
| <b>Evidencia</b>         | Actividad de 3 puntos: video de hasta 3 minutos defendiendo los tres riesgos principales y el alcance elegido.                                                                            |
| <b>Sesión de 120 min</b> | 10 activación; 25 conceptos; 20 demostración; 40 laboratorio; 15 revisión; 10 cierre.                                                                                                     |

## Bloque 2. Criterios de aceptación y diseño sistemático

|                          |                                                                                                                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Derivar pruebas compactas y potentes a partir de requisitos, reglas y estados.                                                                                                              |
| <b>Resultados</b>        | Formulaciones; aplica equivalencia, límites, decisión, estados, pairwise y cobertura estructural básica.                                                                                    |
| <b>Contenidos</b>        | Test basis; condiciones, casos y charters; Given/When/Then como lenguaje; particiones; fronteras; tablas de decisión; transiciones; pairwise; caja blanca; cobertura de sentencias y ramas. |
| <b>Demostración</b>      | Diseño de una matriz de prueba para registro, descuentos o permisos con reglas combinadas.                                                                                                  |
| <b>Práctica</b>          | Reducir un espacio grande de entradas sin perder riesgos relevantes y revisar trazabilidad.                                                                                                 |
| <b>Tecnologías</b>       | Plantillas tabulares; herramienta pairwise opcional; repositorio Markdown.                                                                                                                  |
| <b>Canvas LMS</b>        | Página de técnicas, ejemplos resueltos, laboratorio, banco de requisitos y checklist.                                                                                                       |
| <b>Evidencia</b>         | Actividad de 3 puntos: matriz de casos con explicación de técnica, formulación y riesgo cubierto.                                                                                           |
| <b>Sesión de 120 min</b> | 10 activación; 25 conceptos; 20 demostración; 45 práctica; 10 revisión; 10 cierre.                                                                                                          |

### Bloque 3. Pruebas unitarias y de componentes con TypeScript y Vitest

|                          |                                                                                                                                                                                |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Crear una suite rápida, legible y aislada que proteja reglas de negocio.                                                                                                       |
| <b>Resultados</b>        | Organiza pruebas AAA; valida casos normales y bordes; maneja asincronía, fixtures, mocks, spies y snapshots con criterio.                                                      |
| <b>Contenidos</b>        | Estructura del proyecto; test runner, assertions y matchers; parametrización; setup/teardown; dobles; errores; tiempo; propiedades; componentes; legibilidad y mantenibilidad. |
| <b>Demostración</b>      | Regla de negocio con dependencias y estados de error; refactor de una prueba frágil.                                                                                           |
| <b>Práctica</b>          | Implementar pruebas para un módulo TypeScript y justificar que no debe simularse.                                                                                              |
| <b>Tecnologías</b>       | TypeScript, Node.js, Vitest y Git.                                                                                                                                             |
| <b>Canvas LMS</b>        | Guía de entorno, repositorio base, laboratorio, catálogo de errores y tarea con rúbrica.                                                                                       |
| <b>Evidencia</b>         | Actividad de 3 puntos: suite ejecutable, repositorio y video de hasta 3 minutos explicando una decisión de aislamiento.                                                        |
| <b>Sesión de 120 min</b> | 10 activación; 20 conceptos; 25 demostración; 45 laboratorio; 10 revisión; 10 cierre.                                                                                          |

### Bloque 4. TDD, calidad de la suite y CI inicial

|                          |                                                                                                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Usar pruebas para guiar diseño y medir su capacidad real de detectar cambios.                                                                                      |
| <b>Resultados</b>        | Aplica red-green-refactor; interpreta cobertura sin confundirla con efectividad; ejecuta mutación y configura un quality gate inicial.                             |
| <b>Contenidos</b>        | TDD; test smells; pirámide/diamante; cobertura; mutation score; determinismo; duración; flakiness; revisión; workflow, caché, artefactos y checks de pull request. |
| <b>Demostración</b>      | Ciclo TDD y mutación sobreviviente; pipeline que publica reporte y bloquea un cambio defectuoso.                                                                   |
| <b>Práctica</b>          | Mejorar una suite que tiene cobertura alta pero detecta pocos mutantes.                                                                                            |
| <b>Tecnologías</b>       | Vitest, Stryker, GitHub Actions y GitHub pull requests.                                                                                                            |
| <b>Canvas LMS</b>        | Lectura guiada, laboratorio TDD, checklist de suite, guía de pipeline y evaluación del módulo.                                                                     |
| <b>Evidencia</b>         | Cierre de módulo: estrategia, suite, mutation score y pipeline defendidos con evidencia.                                                                           |
| <b>Sesión de 120 min</b> | 10 activación; 20 conceptos; 25 demostración; 45 laboratorio; 10 revisión; 10 cierre.                                                                              |

## MÓDULO 2

## Integración y automatización de flujos

De dependencias reales a contratos, API y recorridos End-to-End con evidencia.

## Bloque 5. Integración reproducible con PostgreSQL y Testcontainers

|                          |                                                                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Validar límites reales entre código y persistencia sin depender de ambientes compartidos.                                                                              |
| <b>Resultados</b>        | Levanta dependencias efímeras; controla migraciones, semillas, transacciones, limpieza y aislamiento.                                                                  |
| <b>Contenidos</b>        | Integración vs unidad; sociable/solitaria; Docker; lifecycle de contenedores; PostgreSQL; migraciones; fixtures; hermeticidad; concurrencia; fallos de red y limpieza. |
| <b>Demostración</b>      | Prueba de repositorio que inicia PostgreSQL, aplica migración, valida datos y destruye el recurso.                                                                     |
| <b>Práctica</b>          | Convertir una prueba dependiente de una base local en una prueba reproducible.                                                                                         |
| <b>Tecnologías</b>       | TypeScript, Vitest, PostgreSQL, Docker y Testcontainers for Node.js.                                                                                                   |
| <b>Canvas LMS</b>        | Arquitectura, laboratorio, troubleshooting, recursos oficiales y criterios de finalización.                                                                            |
| <b>Evidencia</b>         | Actividad de 3 puntos: integración real reproducible y explicación de aislamiento y limpieza.                                                                          |
| <b>Sesión de 120 min</b> | 10 activación; 20 conceptos; 25 demostración; 50 laboratorio; 5 revisión; 10 cierre.                                                                                   |

## Bloque 6. API testing y contratos OpenAPI

|                          |                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Evaluar comportamiento, estructura y compatibilidad de una API más allá del código de estado.                                                                    |
| <b>Resultados</b>        | Diseña colecciones tiles; valida esquema, errores, idempotencia, autenticación, paginación y compatibilidad.                                                     |
| <b>Contenidos</b>        | HTTP aplicado; pirámide de API; ambientes y secretos; oráculos; datos; pruebas negativas; OpenAPI 3.2; schema validation; property-based/fuzzing como extensión. |
| <b>Demostración</b>      | Exploración de una API y detección de deriva entre implementación y especificación.                                                                              |
| <b>Práctica</b>          | Crear una matriz API y automatizar validaciones funcionales y de esquema.                                                                                        |
| <b>Tecnologías</b>       | Postman, OpenAPI, CLI/Newman o runner equivalente; GitHub Actions.                                                                                               |
| <b>Canvas LMS</b>        | Especificación, colección base, laboratorio, checklist de API y tarea.                                                                                           |
| <b>Evidencia</b>         | Actividad de 3 puntos: colección, casos negativos, evidencia de ejecución y video de hasta 3 minutos.                                                            |
| <b>Sesión de 120 min</b> | 10 activación; 25 conceptos; 20 demostración; 45 laboratorio; 10 revisión; 10 cierre.                                                                            |

## Bloque 7. Contract testing con Pact

|                          |                                                                                                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Detectar cambios incompatibles entre servicios sin depender de pruebas E2E lentas.                                                                      |
| <b>Resultados</b>        | Distingue schema y consumer-driven contracts; publica/verifica pactos y razona sobre compatibilidad.                                                    |
| <b>Contenidos</b>        | Consumidor/proveedor; interacciones; pactos; broker; provider states; verificación; versionado; compatibilidad; deployment checks; límites del enfoque. |
| <b>Demostración</b>      | Cambio de proveedor que conserva OpenAPI pero rompe una expectativa real del consumidor.                                                                |
| <b>Práctica</b>          | Crear un pacto pequeño, verificarlo y documentar el cambio incompatible.                                                                                |
| <b>Tecnologías</b>       | Pact para JavaScript/TypeScript, Pact Broker opcional y GitHub Actions.                                                                                 |
| <b>Canvas LMS</b>        | Diagrama, laboratorio consumidor, laboratorio proveedor, guía de errores y foro de discusión.                                                           |
| <b>Evidencia</b>         | Actividad de 3 puntos: pacto verificado y explicación del riesgo que elimina y el que no elimina.                                                       |
| <b>Sesión de 120 min</b> | 10 actividades; 25 conceptos; 25 demostraciones; 40 laboratorio; 10 revisión; 10 cierre.                                                                |

## Bloque 8. E2E web, regresión visual y pipeline ampliado

|                          |                                                                                                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Propósito</b>         | Automatizar recorridos críticos con evidencia rica, minimizando fragilidad y costo de mantenimiento.                                                                              |
| <b>Resultados</b>        | Selecciona recorridos; usa locators resistentes, fixtures, trazas y screenshots; compara Playwright y Cypress.                                                                    |
| <b>Contenidos</b>        | Modelo de aislamiento; accesibilidad de locators; Page Objects/fixtures con criterio; paralelismo; sharding; redes; visual comparison; retries responsables; artefactos y triage. |
| <b>Demostración</b>      | Flujo crítico multi-navegador con trace viewer y diferencia visual controlada.                                                                                                    |
| <b>Práctica</b>          | Automatizar happy path y fallo significativo; diagnosticar una prueba inestable.                                                                                                  |
| <b>Tecnologías</b>       | Playwright como ruta principal; Cypress como comparación; GitHub Actions.                                                                                                         |
| <b>Canvas LMS</b>        | Guía E2E, repositorio, laboratorio de trazas, checklist de flakiness y evaluación del módulo.                                                                                     |
| <b>Evidencia</b>         | Cierre de módulo: integración, API/contrato y E2E ejecutados en CI con reportes y trazas.                                                                                         |
| <b>Sesión de 120 min</b> | 10 actividades; 20 conceptos; 25 demostraciones; 45 laboratorio; 10 triage; 10 cierre.                                                                                            |

## MÓDULO 3

## Calidad avanzada y operación

Atributos no funcionales, observabilidad, métricas y sistemas con IA.

## Bloque 9. Rendimiento y confiabilidad con k6

|                   |                                                                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Propósito         | Modelar carga realista y convertir objetivos de servicio en umbrales verificables.                                                                                                 |
| Resultados        | Distingue carga, estrés, spike y endurance; define escenarios, métricas, percentiles, checks y thresholds.                                                                         |
| Contenidos        | Workload model; usuarios virtuales y tasa de llegada; warm-up; p90/p95/p99; throughput; errores; saturación; SLI/SLO; baseline; entorno; lectura de resultados y límites críticos. |
| Demostración      | Prueba k6 que falla un umbral y correlación con señales del sistema.                                                                                                               |
| Práctica          | Diseñar un perfil pequeño, ejecutar baseline y explicar el cuello de botella sin sobreafirmar.                                                                                     |
| Tecnologías       | k6, API de prueba, GitHub Actions y telemetría disponible.                                                                                                                         |
| Canvas LMS        | Guía de carga segura, dataset, laboratorio, plantilla de reporte y criterios de autorización.                                                                                      |
| Evidencia         | Actividad de 3 puntos: script, umbrales, gráfica/reporte y video interpretando un resultado.                                                                                       |
| Sesión de 120 min | 10 activación; 25 conceptos; 20 demostración; 45 laboratorio; 10 análisis; 10 cierre.                                                                                              |

## Bloque 10. Seguridad, accesibilidad y calidad visual

|                   |                                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Propósito         | Verificar atributos críticos que no pueden reducirse a pruebas funcionales felices.                                                                                       |
| Resultados        | Deriva verificaciones de OWASP ASVS; combina pruebas automáticas y manuales de WCAG; controla regresión visual.                                                           |
| Contenidos        | Autenticación, autorización, validación, sesiones, secretos y dependencias; WCAG 2.2 y ACT; teclado, foco, nombre/rol/valor; contraste; baselines visuales y tolerancias. |
| Demostración      | Revisión de un flujo con falla de control de acceso, barrera de teclado y cambio visual inesperado.                                                                       |
| Práctica          | Aplicar checklists, automatizar una parte y documentar los límites de la automatización.                                                                                  |
| Tecnologías       | OWASP ASVS 5.0, Playwright/axe opcional, snapshots ARIA y visual comparison.                                                                                              |
| Canvas LMS        | Laboratorio vulnerable, checklist ASVS, guía WCAG, prueba manual y revisión de hallazgos.                                                                                 |
| Evidencia         | Actividad de 3 puntos: tres hallazgos priorizados, pasos de reproducción y propuesta de verificación.                                                                     |
| Sesión de 120 min | 10 activación; 30 conceptos; 20 demostración; 40 auditoría; 10 revisión; 10 cierre.                                                                                       |

## Bloque 11. M vil, observabilidad, m tricas y suites confiables

|                          |                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prop sito</b>         | Diagnosticar calidad en ejecuci n y mantener suites que aporten se al estable.                                                                                                         |
| <b>Resultados</b>        | Automatiza un flujo m vil; correlaciona logs, m tricas y trazas; mide flakiness, duraci n y defectos escapados.                                                                        |
| <b>Contenidos</b>        | Flujos m viles y permisos; Maestro; pir mide m vil; telemetr a; OpenTelemetry; trazas y spans; m tricas de suite; flaky tests; cuarentena temporal; ownership; defect lifecycle y RCA. |
| <b>Demostraci n</b>      | Fallo E2E m vil investigado con evidencia y traza del backend; tablero m nimo de salud de pruebas.                                                                                     |
| <b>Pr ctica</b>          | Crear flujo m vil o simulaci n equivalente y diagnosticar un fallo con se ales correlacionadas.                                                                                        |
| <b>Tecnolog as</b>       | Maestro, OpenTelemetry, reportes de CI y tablero simple.                                                                                                                               |
| <b>Canvas LMS</b>        | Setup m vil, laboratorio, dataset de incidente, plantilla RCA y dashboard de m tricas.                                                                                                 |
| <b>Evidencia</b>         | Actividad de 3 puntos: automatizaci n o diagn stico reproducible con evidencia correlacionada.                                                                                         |
| <b>Sesi n de 120 min</b> | 10 activaci n; 25 conceptos; 20 demostraci n; 45 laboratorio; 10 revisi n; 10 cierre.                                                                                                  |

## Bloque 12. Pruebas de sistemas con IA y cierre integrador

|                          |                                                                                                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prop sito</b>         | Evaluar sistemas variables y defender una estrategia de calidad integral con l mites expl citos.                                                                                                         |
| <b>Resultados</b>        | Dise a conjuntos de evaluaci n; aplica r bricas, repetici n y supervisi n; eval a seguridad, robustez y trazabilidad.                                                                                    |
| <b>Contenidos</b>        | No determinismo; golden sets; jueces humanos/automatizados; m tricas; variaci n; groundedness; alucinaci n; prompt injection; privacidad; sesgo; red teaming; regresi n de prompts/modelos; NIST AI RMF. |
| <b>Demostraci n</b>      | Evaluaci n repetida de una funci n con IA y an lisis de dispersi n, errores cr ticos y falsos positivos del juez.                                                                                        |
| <b>Pr ctica</b>          | Crear un peque o conjunto de evaluaci n, una r brica y un reporte con limitaciones.                                                                                                                      |
| <b>Tecnolog as</b>       | Herramienta de IA autorizada, scripts de evaluaci n, repositorio y CI opcional.                                                                                                                          |
| <b>Canvas LMS</b>        | Pol tica de IA, caso, plantilla de eval, checklist de seguridad, entrega y defensa final.                                                                                                                |
| <b>Evidencia</b>         | Cierre de m dulo y proyecto: quality gate integral, informe de riesgos, demo y defensa t cnica.                                                                                                          |
| <b>Sesi n de 120 min</b> | 10 activaci n; 25 conceptos; 25 demostraci n; 40 laboratorio; 10 revisi n; 10 cierre.                                                                                                                    |

## 10. Proyecto integrador del curso

El proyecto consiste en asegurar la calidad de una aplicación moderna con interfaz, API y persistencia, o de un sistema equivalente autorizado por el docente. Puede ser individual o grupal según la oferta. El producto no necesita ser grande: debe ser observable, reproducible y suficientemente rico para justificar decisiones en varias capas.

### 10.1 Hitos progresivos

| Hito                       | Alcance mínimo                                                                                              | Evidencia                                                     |
|----------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| A. Charter y estrategia    | Contexto, usuarios, riesgos, atributos, criterios de aceptación, niveles, ambientes y plan de datos.        | Quality charter, matriz riesgo-prueba y backlog trazable.     |
| B. Código y componentes    | Suite unitaria/componentes, TDD selectivo, cobertura interpretada, mutation testing y CI inicial.           | Repositorio, reporte, mutantes analizados y workflow.         |
| C. Integración y contratos | PostgreSQL efímero, migraciones, pruebas de integración, API/OpenAPI y al menos un contrato cuando aplique. | Ejecución reproducible, colección/reporte y pacto verificado. |
| D. Flujos y no funcional   | E2E crítico, evidencia visual, rendimiento, seguridad, accesibilidad y móvil si corresponde.                | Trazas, screenshots, umbrales y hallazgos priorizados.        |
| E. Operación y cierre      | Métricas de suite, observabilidad disponible, riesgos residuales, deuda y quality gate final.               | Dashboard o resumen, informe ejecutivo, demo y defensa.       |

### 10.2 Requisitos mínimos del repositorio

README con propósito, requisitos, instalación, ejecución, estructura y decisiones principales.

Scripts reproducibles para unit, integration, contract, e2e y las pruebas no funcionales incluidas.

Variables de entorno documentadas con archivo de ejemplo, sin secretos ni datos personales.

Datos sintéticos o anonimizados, migraciones versionadas y estrategia explícita de limpieza.

Pipeline con checks relevantes, reportes y artefactos de fallos; no se aceptan reintentos ciegos como solución a flakiness.

TEST\_STRATEGY.md, AI\_USAGE.md, registro de riesgos y evidencia de defectos/hallazgos.

Riesgos residuales y límites: qué no se proba, por qué y qué evidencia se requiere.

### 10.3 Quality gate recomendado

| Puerta                 | Regla mínima                                                                           | Decisión                                      |
|------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------|
| Compilación y análisis | Tipos y verificaciones estáticas sin errores bloqueantes.                              | Bloquea integración.                          |
| Pruebas rápidas        | Suite unitaria/componentes estable y dentro del presupuesto de tiempo.                 | Bloquea integración.                          |
| Integración/contrato   | Dependencias efímeras, migraciones y contratos verificados.                            | Bloquea integración si aplica al cambio.      |
| E2E crítico            | Recorridos priorizados con artefactos de fallo y sin flakiness conocida no gestionada. | Bloquea release o exige aceptación de riesgo. |
| No funcional           | Umbrales acordados y cero hallazgos críticos abiertos.                                 | Bloquea release.                              |
| Trazabilidad           | Requisito/riesgo, prueba, ejecución y defecto conectados.                              | Revisión antes de la defensa.                 |

## 11. Evaluación

La distribución siguiente es una propuesta adaptable a las políticas institucionales. Conserva tres evaluaciones principales, una al cierre de cada módulo, sin asignarlas a semanas. La mayoría de las tareas breves vale 3 puntos.

| Componente              | Puntos | Descripción                                                                               |
|-------------------------|--------|-------------------------------------------------------------------------------------------|
| Ocho tareas breves      | 24     | Evidencias de 3 puntos con video de hasta 3 minutos, repositorio y resultado verificable. |
| Evaluación del módulo 1 | 10     | Caso práctico de riesgo, diseño y pruebas unitarias con lectura crítica de evidencia.     |
| Evaluación del módulo 2 | 15     | Caso integrado de datos, API/contrato, E2E y pipeline.                                    |
| Evaluación del módulo 3 | 15     | Caso de calidad no funcional, observabilidad o evaluación de un sistema con IA.           |
| Proyecto integrador     | 30     | Hitos, quality gate, informe, demostración y defensa.                                     |
| Portafolio y reflexión  | 6      | Índice de evidencias, retroalimentación aplicada y autoevaluación de competencias.        |
| Total                   | 100    | Calificación total recomendada.                                                           |

### 11.1 Principios de evaluación

**Autenticidad:** el estudiante debe poder ejecutar, explicar y modificar su solución.

**Trazabilidad:** toda afirmación técnica importante debe conectarse con evidencia observable.

**Proporcionalidad:** se evalúa el riesgo y el razonamiento, no la cantidad de herramientas o líneas de código.

**Reproducibilidad:** otra persona debe poder ejecutar la evidencia con instrucciones suficientes.

**Retroalimentación:** las revisiones deben producir una acción concreta para la siguiente entrega.

**Accesibilidad:** se permiten alternativas equivalentes cuando exista una necesidad documentada, sin reducir el resultado de aprendizaje.

### 11.2 Rbrica estándar para tareas de 3 puntos

| Criterio                     | Puntos | Desempeño esperado                                                                          |
|------------------------------|--------|---------------------------------------------------------------------------------------------|
| Corrección técnica           | 1.00   | La solución cumple el objetivo, cubre el riesgo principal y maneja los estados solicitados. |
| Evidencia y reproducibilidad | 0.75   | Repositorio/enlace, instrucciones, ejecución y artefactos permiten verificar el resultado.  |
| Razonamiento y decisiones    | 0.75   | Explicación técnica, oráculo, alcance, limitaciones y una decisión relevante.               |
| Comunicación y formato       | 0.50   | Video claro de hasta 3 minutos, entrega organizada y enlaces accesibles.                    |
| Total                        | 3.00   | La rbrica se publica en Canvas antes de la actividad.                                       |

### 11.3 Formato de entrega de tareas breves

#### Nota de implementación

La regla evita una descalificación automática sin revisar el resto de la evidencia, pero mantiene el video como parte obligatoria del resultado de aprendizaje y asigna 0 en los criterios que no puedan demostrarse.

Un único envío en Canvas con nombre, carné, enlace al repositorio o artefacto, enlace al video y evidencia solicitada.

Video de máximo 3 minutos: 20 segundos de contexto, 100 segundos de demostración y 60 segundos de explicación/limitaciones, como guá flexible.

El video debe mostrar ejecución real y no solo diapositivas. No se deben exponer credenciales, tokens, datos personales ni secretos.



Los enlaces deben ser accesibles para el docente hasta el cierre del proceso de revisión. Se recomienda verificar permisos en una ventana privada.

Si falta el video, la entrega no demuestra el resultado de comunicación requerido y se califica con la evidencia disponible según la rbrica; las excepciones por accesibilidad se coordinan previamente.

## 12. Política de uso de inteligencia artificial

Las herramientas de IA pueden emplearse como asistentes de aprendizaje y productividad cuando su uso sea autorizado. El estudiante conserva la responsabilidad por exactitud, seguridad, licencias, privacidad, comprensión y defensa del trabajo.

| Categoría    | Permitido con trazabilidad                                                          | No permitido                                                                                 |
|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Exploración  | Proponer riesgos, escenarios, casos borde, datos sintéticos o alternativas.         | Sustituir el análisis del caso o copiar una matriz sin revisarla.                            |
| Código       | Generar scaffolding, ejemplos pequeños o revisiones que luego se prueban y adaptan. | Entregar código que no se puede explicar, ejecutar o modificar.                              |
| Diagnóstico  | Explicar errores, resumir logs y sugerir hipótesis que se verifican con evidencia.  | Fabricar ejecuciones, reportes, capturas, métricas o conclusiones.                           |
| Datos        | Crear datos sintéticos sin información real sensible.                               | Compartir credenciales, secretos, información personal o material institucional restringido. |
| Evaluaciones | Uso definido expresamente por el docente para la actividad.                         | Usar IA en evaluaciones cerradas o defensas cuando no está autorizado.                       |
| Autoría      | Declarar herramienta/modelo, fecha, propósito, aportes aceptados y correcciones.    | Ocultar el uso o atribuir a la IA responsabilidad por errores.                               |

### 12.1 Registro AI\_USAGE.md

Cada proyecto que use IA incluye un registro breve con: herramienta y modelo cuando se conozca; fecha; propósito; resumen del prompt sin datos sensibles; salida aceptada, modificada o rechazada; verificación realizada; riesgos o limitaciones. No es necesario copiar conversaciones completas.

### 12.2 Criterios de defensa

Explicar la relación entre riesgo, técnica, caso y oráculo.

Modificar una parte acotada o predecir el efecto de un cambio.

Mostrar que sugerencia de IA fue incorrecta o insuficiente y cómo se detectó.

Identificar datos, permisos y secretos que no deben compartirse.

Diferenciar una prueba que pasa de evidencia suficiente para una decisión.

### 13. Pruebas de sistemas habilitados con IA

Los sistemas con modelos generativos o predictivos requieren un enfoque distinto al de funciones deterministas. Una sola ejecución no caracteriza el comportamiento. El curso introduce evaluación repetida, conjuntos representativos, criterios humanos y automatizados, análisis de variación y revisión de riesgos.

| Dimensión            | Pregunta de prueba                                                     | Evidencia mínima                                                    |
|----------------------|------------------------------------------------------------------------|---------------------------------------------------------------------|
| Calidad de respuesta | La salida satisface el propósito y las restricciones del caso?         | Revisión explícita, ejemplos positivos/negativos y revisión humana. |
| Groundedness         | La respuesta se apoya en la fuente disponible y evita inventar hechos? | Citas/fuentes verificables y casos de información ausente.          |
| Variabilidad         | El comportamiento es aceptable en repeticiones y cambios menores?      | Múltiples ejecuciones, dispersión y tasa de fallos críticos.        |
| Robustez y seguridad | Resiste prompt injection, abuso, jailbreaks o entradas adversarias?    | Casos rojos, severidad, contención y evidencia de mitigación.       |
| Privacidad           | Evita exponer o memorizar datos no autorizados?                        | Dataset sintético, controles, pruebas de fuga y revisión de logs.   |
| Sesgo y cobertura    | El conjunto representa grupos, idiomas y escenarios relevantes?        | Matriz de cobertura, resultados segmentados y límites.              |
| Regresión            | Un cambio de prompt, modelo o herramienta altera capacidades críticas? | Versionado, conjunto estable, comparación y criterio de aceptación. |
| Supervisión          | Las decisiones de alto impacto conservan control humano apropiado?     | Escalamiento, abstención, revisión y registro de decisiones.        |

**Marco de referencia**

El diseño de evaluaciones de IA se apoya en la gestión de riesgos y en prácticas de testing, evaluation, verification and validation (TEVV). Las métricas automáticas o los jueces con IA deben validarse contra criterios humanos y nunca presentarse como verdad absoluta.

## 14. Organizaci n recomendada en Canvas LMS

Canvas debe reflejar la progresi n modular y no una cronolog a fija. Las fechas, disponibilidad y ritmo se configuran en cada oferta sin renombrar el contenido por semanas.

| Orden | M dulo/ rea         | Contenido recomendado                                                                                              |
|-------|---------------------|--------------------------------------------------------------------------------------------------------------------|
| 0     | Comenzar aqu        | Bienvenida, programa, gua, calendario de la oferta, pol tica de IA, netiqueta, accesibilidad, setup y diagn stico. |
| 1     | M dulo 1            | Cuatro bloques de fundamentos, dise o, Vitest y calidad continua; evaluaci n de cierre.                            |
| 2     | M dulo 2            | Cuatro bloques de integraci n, API, contratos y E2E; evaluaci n de cierre.                                         |
| 3     | M dulo 3            | Cuatro bloques de no funcional, seguridad/accesibilidad, operaci n e IA; evaluaci n de cierre.                     |
| 4     | Proyecto integrador | Brief, hitos, r bricas, ejemplos de evidencia, entregas, revisi n y defensa.                                       |
| 5     | Recursos            | Glosario, troubleshooting, documentaci n oficial, repositorios base, plantillas y lecturas opcionales.             |

### 14.1 Plantilla para cada bloque tem tico

| Orden | Elemento               | Prop sito                                                                     |
|-------|------------------------|-------------------------------------------------------------------------------|
| 1     | Encabezado del bloque  | T tulo, relaci n con el proyecto, prerrequisitos y tiempo estimado.           |
| 2     | P gina de introducci n | Pregunta rectora, resultados, vocabulario y evidencia esperada.               |
| 3     | Contenido esencial     | Explicaci n suficiente, diagramas, ejemplos, decisiones y errores frecuentes. |
| 4     | Demostraci n           | Video breve o walkthrough con c digo y evidencia descargable.                 |
| 5     | Laboratorio guiado     | Repositorio, pasos, checkpoints, extensiones y criterios de finalizaci n.     |
| 6     | Recursos oficiales     | Documentaci n seleccionada con prop sito de lectura, no enlaces sin contexto. |
| 7     | Tarea o comprobaci n   | Consigna, r brica de 3 puntos, formato, video y ejemplo m nimo de evidencia.  |
| 8     | Cierre                 | Resumen, exit ticket, troubleshooting y conexi n con el siguiente bloque.     |

### 14.2 Convenciones de dise o y operaci n

Publicar resultados de aprendizaje, criterios y r brica antes de habilitar una entrega.

Usar requisitos de m dulo y prerrequisitos solo cuando representen una dependencia real; evitar bloquear por navegaci n superficial.

Mantener p ginas sem nticas, encabezados jer rquicos, texto alternativo, contraste, subt tulos y enlaces descriptivos.

Conservar repositorios base por bloque y tags/releases para cada oferta; no reemplazar silenciosamente material ya asignado.

Usar grupos y peer review cuando el aprendizaje requiera colaboraci n, pero mantener evidencia individual de comprensi n.

Configurar SpeedGrader con la misma r brica y comentarios frecuentes reutilizables.

Revisar anal tica de acceso, entrega y errores para detectar barreras, sin convertir actividad de plataforma en sustituto del aprendizaje.

Mantener nuevo contenido sin publicar hasta que consigna, enlaces, r brica, permisos y prueba de estudiante hayan sido verificados.

### 14.3 Checklist de publicación

| Antes de publicar | Validación                                                                              |
|-------------------|-----------------------------------------------------------------------------------------|
| Contenido         | Título, propósito, resultados, prerequisites, tiempo y conexión con el proyecto.        |
| Laboratorio       | Repositorio clonado desde cero, comandos verificados y errores frecuentes documentados. |
| Entrega           | Puntos, rúbrica, video, enlaces, ejemplo, fechas de la oferta y política de atrasos.    |
| Acceso            | Student View, permisos externos, enlaces en ventana privada y archivos descargables.    |
| Accesibilidad     | Encabezados, contraste, teclado, alt text, captions/transcripción y lenguaje claro.     |
| Integridad        | Sin secretos, datos personales, respuestas de evaluación o materiales no licenciados.   |

## 15. Matriz de actualización respecto a la guía anterior

| Guía anterior                       | Actualización 2026                                                       | Justificación                                                          |
|-------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------|
| Dosificación por semanas            | Tres módulos y doce bloques reordenables.                                | Permite adaptar la oferta sin perder progreso ni resultados.           |
| Fundamentos de sistemas de calidad  | Fundamentos de Quality Engineering, producto/proceso y riesgo.           | Conserva la base y la conecta con decisiones de software.              |
| Administración estratégica amplia   | Estrategia de calidad del producto y quality gates.                      | Evita duplicar administración general y prioriza aplicación técnica.   |
| Niveles y tipos de prueba           | Niveles, tipos, pirámide/diamante y alcance basado en riesgo.            | Mantiene contenido duradero con contexto DevOps.                       |
| Técnicas tradicionales              | Equivalencia, límites, decisión, estados, pairwise y estructura.         | Se conserva y se vincula con criterios de aceptación y automatización. |
| Testing repetido en varias unidades | Bloques con propósito, oráculo, datos, trazabilidad y evidencia.         | Reduce redundancia y aumenta profundidad.                              |
| Control de versiones tardío         | Git y GitHub desde el inicio como infraestructura del aprendizaje.       | Toda evidencia requiere historial y reproducibilidad.                  |
| Automatización genérica             | TypeScript, Vitest, CI, Testcontainers, Pact y Playwright.               | Crea una ruta coherente de menor a mayor alcance.                      |
| Gestión de bugs y depuración        | Defect lifecycle, evidence-rich reports, RCA, métricas y observabilidad. | Moderniza diagnóstico y aprendizaje operacional.                       |
| Técnicas estáticas/dinámicas        | Revisión, tipos, análisis, pruebas dinámicas y quality gates.            | Integra las técnicas en el flujo de entrega.                           |
| Robot Framework como cierre         | Proyecto integral con quality gate y defensa.                            | Prioriza estrategia transferible sobre una herramienta keyword-driven. |
| Bibliografía de 1990-2015           | Normas y documentación oficial vigente, con revisión anual.              | Reduce obsolescencia y mejora verificabilidad.                         |

15.1 Matriz de mejora respecto al curso impartido en 2025

| Curso 2025                                | Mejora 2026                                                                                      | Valor                                                                         |
|-------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| TypeScript, Vitest y GitHub Actions       | Se conservan y se conectan con TDD, mutation testing, artefactos y quality gates.                | La ejecuci n se convierte en criterio de calidad, no solo configuraci n.      |
| GitHub Desktop como entrada               | Git por terminal o interfaz a elecci n; foco en conceptos, historial y PR.                       | Evita dependencia de una interfaz espec fica.                                 |
| PostgreSQL local/Docker                   | Testcontainers como ruta principal para dependencias ef meras; servicios de CI como alternativa. | Mejora aislamiento, paralelismo y reproducibilidad.                           |
| Postman para API y carga                  | Postman/OpenAPI para API funcional; k6 para rendimiento.                                         | Separa or culos, modelos y m tricas diferentes.                               |
| Cypress como E2E web                      | Playwright principal; Cypress como comparaci n v lida.                                           | Ampl a multi-navegador, trazas y evidencia, sin descartar experiencia previa. |
| Maestro m vil                             | Se conserva como bloque o extensi n del proyecto.                                                | Mantiene una pr ctica vigente sin imponer m vil a todos los proyectos.        |
| Tareas de 3 y 4 puntos                    | R brica nica de 3 puntos para evidencias breves.                                                 | Simplifica expectativas y calificaci n.                                       |
| Video obligatorio de 3 minutos            | Se conserva con estructura sugerida y alternativa accesible.                                     | Favorece demostraci n y defensa sin penalizaci n opaca.                       |
| Pocas t cnicas de dise o expl citas       | Criterios, riesgo, equivalencia, l mites, decisi n, estados y pairwise antes de automatizar.     | Evita automatizar casos d biles.                                              |
| Sin contratos expl citos                  | OpenAPI 3.2 y Pact.                                                                              | Detecta deriva y cambios incompatibles entre servicios.                       |
| Cobertura funcional dominante             | Seguridad, accesibilidad, visual, rendimiento, mutaci n y observabilidad.                        | Responde a atributos de calidad del producto y demandas profesionales.        |
| IA como herramienta, sin marco curricular | Pol tica de autor a y bloque de evaluaci n de sistemas con IA.                                   | Trata tanto el uso responsable como la nueva superficie de riesgo.            |

Resultado de la actualizaci n

La nueva propuesta no descarta el trabajo de 2025: lo convierte en una trayectoria m s coherente, transferible y medible, con fundamentos antes de la herramienta y con evidencia til para decisiones de calidad.

## 16. Bibliografía y fuentes oficiales modernas

Se recomienda revisar las versiones vigentes al iniciar cada ciclo académico. Las fuentes de herramientas se usan como documentación de laboratorio; los estándares y marcos sustentan los resultados de aprendizaje. Fecha de consulta: 18 de julio de 2026.

1. Universidad Mariano Gálvez de Guatemala. Guía Didáctica: Aseguramiento de la Calidad del Software, código 048. Documento institucional base.
2. Curso Aseguramiento de la Calidad del Software 2025. Exportación de Canvas LMS utilizada como evidencia de implementación previa.
3. ISTQB. Certified Tester Foundation Level Syllabus v4.0.1. [https://istqb.org/wp-content/uploads/2024/11/ISTQB\\_CTFL\\_Syllabus\\_v4.0.1.pdf](https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf)
4. ISO/IEC 25010:2023. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. <https://www.iso.org/standard/78176.html>
5. TypeScript. Documentación oficial. <https://www.typescriptlang.org/docs/>
6. Vitest. Guía oficial. <https://vitest.dev/guide/>
7. GitHub. GitHub Actions documentation. <https://docs.github.com/en/actions>
8. PostgreSQL Global Development Group. PostgreSQL documentation, current version. <https://www.postgresql.org/docs/current/>
9. Docker. Documentación oficial. <https://docs.docker.com/>
10. Testcontainers. Testcontainers for Node.js. <https://node.testcontainers.org/>
11. Postman. Write test scripts and automate API checks. <https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/>
12. OpenAPI Initiative. OpenAPI Specification, latest published version (3.2.0 al preparar esta guía). <https://spec.openapis.org/oas/latest.html>
13. Pact Foundation. Pact documentation. <https://docs.pact.io/>
14. Microsoft. Playwright documentation. <https://playwright.dev/docs/intro>
15. Cypress. Documentación oficial. <https://docs.cypress.io/>
16. Maestro. Documentación oficial. <https://docs.maestro.dev/>
17. Grafana Labs. k6 documentation. <https://grafana.com/docs/k6/latest/>
18. Stryker Mutator. Mutation testing documentation. <https://stryker-mutator.io/docs/>
19. OWASP Foundation. Application Security Verification Standard 5.0.0. <https://owasp.org/www-project-application-security-verification-standard/>
20. W3C Web Accessibility Initiative. Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>
21. W3C. Accessibility Conformance Testing (ACT) Rules Format. <https://www.w3.org/WAI/standards-guidelines/act/>
22. OpenTelemetry. Concepts and signals: traces, metrics and logs. <https://opentelemetry.io/docs/concepts/signals/>
23. NIST. Artificial Intelligence Risk Management Framework and AI Resource Center. <https://airc.nist.gov/>
24. NIST. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
25. OWASP GenAI Security Project. LLM and Generative AI security resources. <https://genai.owasp.org/>
26. Instructure. Canvas Instructor Guide. <https://community.canvaslms.com/t5/Instructor-Guide/tkb-p/Instructor>

### 16.1 Protocolo de revisión anual

Confirmar versiones estables/LTS de Node.js, TypeScript y dependencias del repositorio base.

Revisar changelogs y guías de migración de Vitest, Playwright, Cypress, Testcontainers, Pact, k6 y Maestro.

Verificar la versión publicada de OpenAPI, OWASP ASVS, WCAG/ACT, ISTQB y marcos NIST de IA.

Ejecutar todos los laboratorios desde un equipo limpio y en GitHub Actions.

Actualizar screenshots, comandos, acciones y políticas de secretos; evitar copiar versiones antiguas de ejemplos previos.

Registrar cambios en una nota de versión de la guía y mantener intacta la evidencia de ofertas anteriores.

## 17. Anexo A - Entorno mínimo recomendado

| Componente    | Recomendación                                                                       | Verificación                                        |
|---------------|-------------------------------------------------------------------------------------|-----------------------------------------------------|
| Sistema       | Windows, macOS o Linux actualizado; virtualización habilitada cuando se use Docker. | Puede ejecutar contenedores y navegador compatible. |
| Runtime       | Node.js LTS compatible con las herramientas seleccionadas.                          | Versiones registradas en README y lockfile.         |
| Editor        | Editor con TypeScript, Git, linting y pruebas; la elección no afecta calificación.  | Proyecto abre sin configuración secreta.            |
| Contenedores  | Docker Desktop, Docker Engine o alternativa compatible aprobada.                    | PostgreSQL efímero inicia y se destruye.            |
| Navegadores   | Motores requeridos por Playwright y navegador para Canvas.                          | Prueba de ejemplo y trace viewer funcionan.         |
| Mail          | Emulador/dispositivo solo para bloques o proyectos que lo requieran.                | Maestro ejecuta un flujo de smoke.                  |
| Cuentas       | GitHub y servicios autorizados por la institución.                                  | MFA habilitado y secretos fuera del repositorio.    |
| Accesibilidad | Subtítulos/transcripción, navegación por teclado y formatos alternativos.           | Materiales y entregas son utilizables.              |

## 18. Anexo B - Checklist para una evidencia de calidad

1. La evidencia comienza con una pregunta, riesgo o criterio de aceptación identificable.
2. La prueba tiene un propósito claro y puede fallar por una razón significativa.
3. Los datos son controlados, sintéticos o anonimizados; la limpieza está documentada.
4. El entorno y las dependencias pueden reproducirse sin pasos ocultos.
5. El resultado incluye logs, reportes, trazas, screenshots o métricas apropiadas al nivel.
6. Las fallas se conservan como evidencia; no se ocultan con reintentos o exclusiones sin explicación.
7. La conclusión distingue hechos observados, inferencias y riesgos residuales.
8. El repositorio no contiene tokens, contraseñas, datos personales ni archivos .env reales.
9. El estudiante puede explicar y modificar el trabajo, haya usado IA o no.
10. La entrega cumple requisitos, permisos, formato y límite de video.

## 19. Anexo C - Glosario esencial

| Término                       | Definición de trabajo                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Aseguramiento de calidad (QA) | Actividades orientadas a generar confianza en que procesos y productos satisfarán necesidades de calidad.                  |
| Control de calidad (QC)       | Técnicas operativas para evaluar productos y detectar desviaciones.                                                        |
| Verificación                  | Comprobar si el producto de trabajo cumple requisitos o especificaciones definidos.                                        |
| Validación                    | Comprobar si el producto satisface necesidades de uso y propósito en su contexto.                                          |
| Error / defecto / fallo       | Acción humana incorrecta / imperfección en un artefacto / manifestación observable incorrecta.                             |
| Oráculo                       | Mecanismo o fuente para decidir si un resultado observado es aceptable.                                                    |
| Fixture                       | Estado, datos o recursos preparados para ejecutar una prueba.                                                              |
| Flaky test                    | Prueba con resultados inconsistentes sin cambio relevante en el sistema bajo prueba.                                       |
| Contract test                 | Verificación de expectativas explícitas entre consumidor y proveedor.                                                      |
| Quality gate                  | Criterio automatizado o revisado que condiciona integración o release.                                                     |
| Observabilidad                | Capacidad de comprender el estado interno mediante señales externas como trazas, métricas y logs.                          |
| Mutation testing              | Evaluación de la suite introduciendo cambios controlados para comprobar si las pruebas los detectan.                       |
| TEVV                          | Testing, evaluation, verification and validation; conjunto de actividades para valorar sistemas, incluido software con IA. |

### Resultado esperado

Al finalizar, el estudiante no solo habrá ejecutado herramientas. Podrá diseñar una estrategia, seleccionar técnicas proporcionales al riesgo, automatizar evidencia en varias capas, interpretar resultados y defender qué nivel de confianza existe y qué incertidumbre permanece.